

Analysis Code Walkthrough

Preoperative muscle morphology and functional recovery after lumbar decompression

Study investigators

2026

Purpose of this document

This document explains the analysis code line by line so that a reader can audit and reproduce every step from the source imaging-and-outcomes dataset to the figures and models reported in the abstract. The code is organised as a small Python package under `src/`. Each module has a single responsibility, and a thin orchestration module (`pipeline.py`) defines the order in which the steps run. No patient data are stored in the repository; the code reads the source workbook from a local path declared in `config.yaml`.

The end-to-end call structure is summarised by the function call graph in Figure 1 and the conceptual design in Figure 2.

Module 1 — `data_loading.py`

This module reads the workbook and converts its awkward two-row header into ordinary column names. The source file labels each anatomical structure on the first header row (for example, “left iliopsoas muscle”) and the per-structure metric on the second row (for example, “Volume (LM cm3”). The data themselves begin on the third row.

```
def load_config(path: str = "config.yaml") -> dict:
    with open(path) as f:
        return yaml.safe_load(f)
```

`load_config` reads the YAML configuration into a dictionary. Centralising the data path and analysis choices here keeps machine-specific and study-specific settings out of the code, which is what allows the repository to remain free of any absolute paths or data.

```
def load_raw(cfg: dict) -> pd.DataFrame:
    raw = pd.read_excel(cfg["DATA_PATH"], sheet_name=cfg["SHEET"], header=None)
```

We read the worksheet with `header=None` so that pandas does not try to interpret either header row as column names; we will build the names ourselves. `cfg["DATA_PATH"]` and `cfg["SHEET"]` come from the configuration, so the data location is never hard-coded.

```
g = raw.iloc[cfg["HEADER_GROUP_ROW"]].ffill()
m = raw.iloc[cfg["HEADER_METRIC_ROW"]]
```

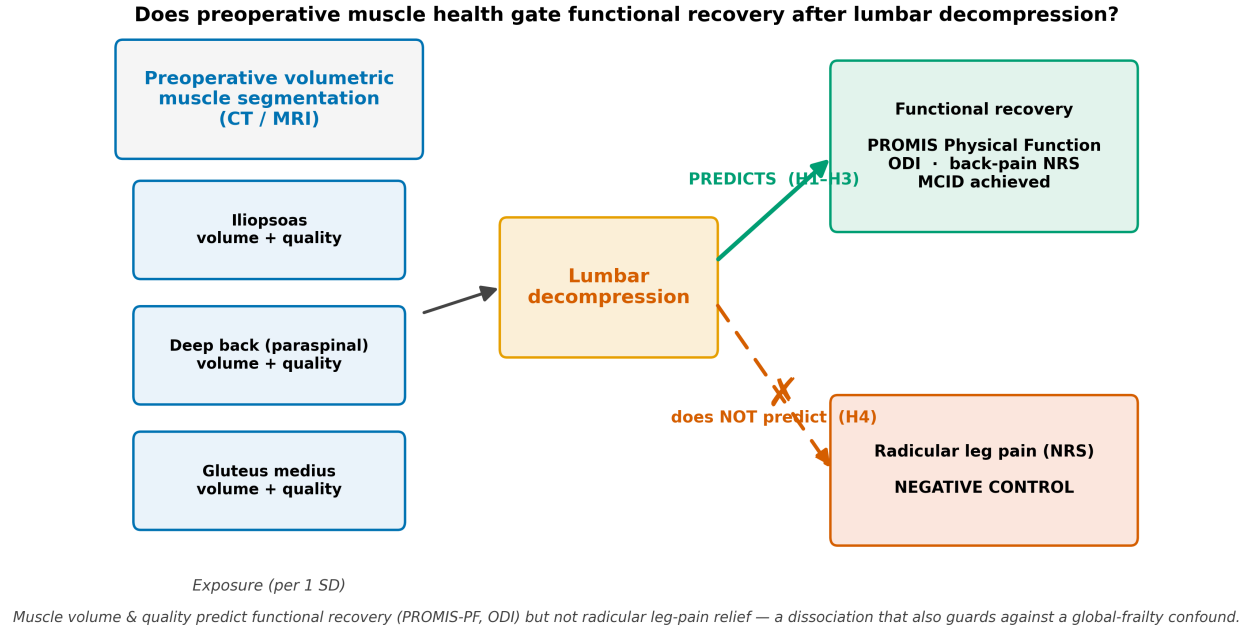



Figure 2: Conceptual study design. Preoperative muscle volume and quality are the exposure; functional outcomes are predicted (solid path), whereas radicular leg pain is a prespecified negative control (dashed path).

The first header row carries the structure label only in its first column and is blank across the remaining columns of that structure's block; `ffill()` (forward fill) propagates each structure label across its block so that every column knows which structure it belongs to. The second row, `m`, holds the metric label for each column.

```
cols = []
for gi, mi in zip(g, m):
    gi = "" if pd.isna(gi) else str(gi).strip()
    mi = "" if pd.isna(mi) else str(mi).strip()
    cols.append((f"{gi} | {mi}").strip(" |"))
```

We pair each structure label with its metric label and join them as "structure | metric", cleaning missing values and stray whitespace. The result is a flat, human-readable, unique-enough column name such as "left iliopsoas muscle | Volume (LM) cm3".

```
data = raw.iloc[cfg["DATA_START_ROW"]:].reset_index(drop=True)
data.columns = cols
return data
```

Finally we drop the two header rows, reset the index, attach the constructed names, and return the analysis frame.

```
def col(data: pd.DataFrame, substr: str) -> pd.Series | None:
    hits = [c for c in data.columns if substr.lower() in c.lower()]
    return data[hits[0]] if hits else None
```

`col` is a convenience accessor: because the flattened names are long, we usually want to fetch a column by a distinctive substring (case-insensitive). It returns the first matching column, or `None` if nothing matches, which keeps downstream code concise.

Module 2 — `cleaning.py`

This module turns raw columns into the variables the models need and addresses the two data-quality issues identified during inspection (volume outliers and ambiguous intensity units). It currently specifies the interface; the bodies are implemented at the analysis stage.

- `derive_demographics` computes age as the interval between date of surgery and date of birth, and body-mass index as weight divided by height in metres squared.
- `winsorize_volumes` caps physiologically implausible muscle volumes. Inspection found an iliopsoas maximum of 170,583 cm³ against a median near 88 cm³, indicating segmentation or data-entry errors; capping the upper tail prevents a handful of artefacts from dominating the regression.
- `build_muscle_exposures` sums left and right sides for each muscle, forms the spine-normalised volume index (muscle volume divided by vertebral-body volume), and standardises each exposure so that effects are interpretable per standard deviation.
- `build_outcomes` extracts the baseline, six-week, and one-year values of each outcome, computes change scores, and flags attainment of the minimal clinically important difference.

Module 3 — `cohort.py`

`build_analytic_cohort` restricts the data to patients who have both a muscle segmentation and the requested outcome timepoint, which is the binding constraint on sample size (approximately 120 to 170 patients). `flow_counts` returns the number of patients remaining at each step so that the patient-flow (STROBE) diagram reports exact exclusions.

Module 4 — `analysis.py`

This module contains the statistical models. `linear_change_model` fits a multivariable linear regression of an outcome change score on a muscle exposure, adjusting for age, sex, body-mass index, number of operated levels, and the baseline outcome value, with robust standard errors; it returns the per-standard-deviation coefficient, its confidence interval, and the p-value. `mcid_logistic` fits the corresponding logistic regression for attainment of the minimal clinically important difference. `run_all` applies these models across the three muscles and both exposure types and assembles a tidy results table, and `sensitivity` repeats the primary analysis under the alternative specifications named in the analysis plan.

Module 5 — `figures.py`

`forest_plot` displays the per-standard-deviation associations of each muscle exposure with the change in physical function, alongside the leg-pain negative control, so that the dissociation is visible in a single panel. `trajectory_plot` shows physical-function recovery over time stratified by muscle quality, and `strobe_diagram` renders the patient-flow counts.

Module 6 — `pipeline.py`

This module wires the others together and defines the order of execution. `prepare` loads the configuration and data and then applies, in sequence, the demographic derivations, volume winsorising, exposure construction, and outcome construction. `run` calls `prepare`, computes the flow counts, builds the analytic cohort, fits all models and sensitivity analyses, and finally writes the figures. Reading `prepare` and `run` top to bottom therefore gives the complete analysis in the exact order it executes, which is the same order shown in the call graph.

Utility modules — `make_schematic.py` and `callgraph.py`

`make_schematic.py` draws the conceptual figure in Figure 2 using only plotting primitives, so it carries no patient data and is fully reproducible. `callgraph.py` parses the package with Python’s `ast` module, records every call from one package-defined function to another, and renders Figure 1; it also writes a Mermaid version (`callgraph.mmd`) that renders directly on the repository page.

How to reproduce

After installing the dependencies and copying `config.example.yaml` to `config.yaml` with the local data path set, the entire analysis runs with:

```
python -m src.pipeline      # full analysis: tables and figures
python src/callgraph.py     # regenerate the call graph
python src/make_schematic.py
```